

Chapter 5

Object Oriented System Design

System Design

- System Design is the process of designing the architecture, components, and interfaces for a system so that it meets the end-user requirements.
- It involves translating user requirements into a detailed blueprint that guides the implementation phase.
- The goal is to create a well-organized and efficient structure that meets the intended purpose while considering factors like scalability, maintainability, and performance.

- System design is decomposed into several activities, each addressing part of the overall problem of decomposing the system.
 - ***Identify design goals.*** Developers identify and prioritize the qualities of the system that they should optimize.
 - ***Design the initial subsystem decomposition.*** Developers decompose the system into smaller parts based on the use case and analysis models. Developers use standard architectural styles as a starting point during this activity.
 - ***Refine the subsystem decomposition to address the design goals.*** The initial decomposition usually does not satisfy all design goals. Developers refine it until all goals are satisfied.

An overview of system design

- Analysis results in the requirements model described by the following products:
 - a set of *nonfunctional requirements* and *constraints*, such as maximum response time, minimum throughput, reliability, operating system platform, and so on
 - a *use case model*, describing the system functionality from the actors' point of view
 - an *object model*, describing the entities manipulated by the system
 - a *sequence diagram* for each use case, showing the sequence of interactions among objects participating in the use case.

Cont..

- The design goals are derived from the nonfunctional requirements. Design goals guide the decisions to be made by the developers when trade-offs are needed.
- The subsystem decomposition constitutes the bulk of system design.
- Developers divide the system into manageable pieces to deal with complexity:
 - each subsystem is assigned to a team and realized independently

System design concepts.

Subsystems and Classes

- In order to reduce the complexity of the application domain, we identified smaller parts called “classes” and organized them into packages.
- Similarly, to reduce the complexity of the solution domain, we decompose a system into simpler parts, called “subsystems,” which are made of a number of solution domain classes.
- A **subsystem** is a replaceable part of the system with well-defined interfaces that encapsulates the state and behavior of its contained classes.

- A subsystem typically corresponds to the amount of work that a single developer or a single development team can tackle.
- By decomposing the system into relatively independent subsystems, concurrent teams can work on individual subsystems with minimal communication overhead.
- In the case of complex subsystems, we recursively apply this principle and decompose a subsystem into simpler subsystems.

Cont..

Services and Subsystem Interfaces

- A subsystem is characterized by the services it provides to other subsystems.
- A **service** is a set of related operations that share a common purpose.
- A subsystem providing a notification service, for example, defines operations to send notices, look up notification channels, and subscribe and unsubscribe to a channel.
- The set of operations of a subsystem that are available to other subsystems form the **subsystem interface**.
- The subsystem interface includes the name of the operations, their parameters, their types, and their return values.
- System design focuses on defining the services provided by each subsystem that is, enumerating the operations, their parameters, and their high-level behavior.

- **Object design** will focus on the **application programmer interface** (API), which refines and extends the subsystem interfaces.
 - The API also includes the type of the parameters and the return value of each operation.

Coupling and Cohesion

- **Coupling** is the number of dependencies between two subsystems.
 - If two subsystems are loosely coupled, they are relatively independent, so modifications to one of the subsystems will have little impact on the other.
 - If two subsystems are strongly coupled, a modification to one subsystem is likely to have impact on the other.
- **Cohesion** is the number of dependencies within a subsystem.
 - If a subsystem contains many objects that are related to each other and perform similar tasks, its cohesion is high.
 - If a subsystem contains a number of unrelated objects, its cohesion is low. A desirable property of subsystem decomposition is that it leads to subsystems with high cohesion.

System design activities: From objects to subsystems

- System design consists of transforming the analysis model into the design model that takes into account the nonfunctional requirements described in the requirements analysis document.

Documenting system design

- System design is documented in the System Design Document (SDD). It describes design goals set by the project, subsystem decomposition (with UML class diagrams), hardware/software mapping (with UML deployment diagrams), data management, access control, control flow mechanisms, and boundary conditions.
- The SDD is used to define interfaces between teams of developers and serve as a reference when architecture-level decisions need to be revisited.
- The audience for the SDD includes the project management, the system architects (i.e., the developers who participate in the system design), and the developers who design and implement each subsystem.

- The first section of the SDD is an ***Introduction***. Its purpose is to provide a brief overview of the software architecture and the design goals. It also provides references to other documents and traceability information.
- The second section, ***Current software architecture***, describes the architecture of the system being replaced. If there is no previous system, this section can be replaced by a survey of current architectures for similar systems.
 - The purpose of this section is to make explicit the background information that system architects used, their assumptions, and common issues the new system will address.

- The third section, ***Proposed system architecture***, documents the system design model of the new system. It is divided into seven subsections:
 - *Overview* presents a bird's-eye view of the software architecture and briefly describes the assignment of functionality to each subsystem.
 - *Subsystem decomposition* describes the decomposition into subsystems and the responsibilities of each. This is the main product of system design.
 - *Hardware/software mapping* describes how subsystems are assigned to hardware and off-the-shelf components. It also lists the issues introduced by multiple nodes and software reuse.
 - *Persistent data management* describes the persistent data stored by the system and the data management infrastructure required for it.
 - This section typically includes the description of data schemes, the selection of a database, and the description of the encapsulation of the database.

..cont

Proposed system architecture

- *Access control and security* describes the user model of the system in terms of an access matrix. This section also describes security issues, such as the selection of an authentication mechanism, the use of encryption, and the management of keys.
- *Global software control* describes how the global software control is implemented. In particular, this section should describe how requests are initiated and how subsystems synchronize. This section should list and address synchronization and concurrency issues.
- *Boundary condition* describes the start-up, shutdown, and error behavior of the system. (If new use cases are discovered for system administration, these should be included in the requirements analysis document, not in this section.)

- The fourth section, ***Subsystem services***, describes the services provided by each subsystem.
 - Although this section is usually empty or incomplete in the first versions of the SDD, this section serves as a reference for teams for the boundaries between their subsystems.
 - The interface of each subsystem is derived from this section and detailed in the Object Design Document.
-
- ☐ The SDD, moreover, is updated throughout the process when design decisions are made or problems are discovered.
 - ☐ The SDD, once published, is baselined and put under configuration management

System Design Document

1. Introduction
 - 1.1 Purpose of the system
 - 1.2 Design goals
 - 1.3 Definitions, acronyms, and abbreviations
 - 1.4 References
 - 1.5 Overview
 2. Current software architecture
 3. Proposed software architecture
 - 3.1 Overview
 - 3.2 Subsystem decomposition
 - 3.3 Hardware/software mapping
 - 3.4 Persistent data management
 - 3.5 Access control and security
 - 3.6 Global software control
 - 3.7 Boundary conditions
 4. Subsystem services
 - Glossary
-

object design

An overview of object design

- Conceptually, software system development fills the gap between a given problem and an existing machine.
- The activities of system development incrementally close this gap by identifying and defining objects that realize part of the system.
- The **object design model** can be partitioned into sets of classes that can be implemented by individual developers.

Object design includes four groups of activities

- ❑ **Reuse.** Off-the-shelf components identified during system design are used to help in the realization of each subsystem.
 - Class libraries and additional components are selected for basic data structures and services.
 - Design patterns are selected for solving common problems and for protecting specific classes from future change.
Often, components and design patterns need to be adapted before they can be used.
 - This is done by rapping custom objects around them or by refining them using inheritance.
- During all these activities, the developers are faced with the same buy-versus-build trade-offs they encountered during system design

Cont..

Object design

- ❑ **Interface specification.** During this activity, the subsystem services identified during system design are specified in terms of class interfaces, including operations, arguments, type signatures, and exceptions.
 - Additional operations and objects needed to transfer data among subsystems are also identified.
 - The result of service specification is a complete interface specification for each subsystem.
 - The subsystem service specification is often called subsystem **API** (Application Programmer Interface)

- ❑ **Restructuring.** Restructuring activities manipulate the system model to increase code reuse or meet other design goals.
 - Each restructuring activity can be seen as a graph transformation on subsets of a particular model.

Cont..

Restructuring

- Typical activities include transforming merging two similar classes from two different subsystems into a single class, splitting complex classes into simpler ones, and/or rearranging classes and operations to increase the inheritance and packaging.
- address design goals such as maintainability, readability, and understandability of the system model.

❑ ***Optimization.*** activities address performance requirements of the system model.

- This includes changing algorithms to respond to speed or memory requirements, reducing multiplicities in associations to speed up queries, adding redundant associations for efficiency, rearranging execution orders, adding derived attributes to improve the access time to objects, and opening up the architecture, that is, adding access to lower layers because of performance requirements.

Cont..

Optimization

- Object design is not sequential. Although each group of activities described above addresses a specific object design issue, they usually occur concurrently.
- during object design, we want the software to be modifiable and extensible to minimize the cost of future changes.

Managing object design

- There are two primary management challenges during object design:
 - ***Increased communication complexity.*** The number of participants involved during this phase of development increases dramatically. The object design models and code are the result of the collaboration of many people.
 - ✓ Management needs to ensure that decisions among these developers are made consistently with project goals.
 - ***Consistency with prior decisions and documents.*** Developers often do not appreciate completely the consequences of analysis and system design decisions before object design.
 - ✓ When detailing and refining the object design model, developers may question some of these decisions and reevaluate them.

Documenting object design

- It describes object design trade-offs made by developers, guidelines they followed for subsystem interfaces, the decomposition of subsystems into packages and classes, and the class interfaces.
- The ODD is used to exchange interface information among teams and as a reference during testing.
- The audience for the ODD includes system architects (i.e., the developers who participate in the system design), developers who implement each subsystem, and testers.
- There are three main approaches to documenting object design:

1. Self-contained ODD generated from model.

- The first approach is to document the object design model the same way we documented the analysis model or the system design model: we write and maintain a UML model and generate the document automatically.
 - This document would duplicate any application objects identified during analysis.
- The disadvantages of this solution include redundancy with the Requirements Analysis Document (RAD) and a high level of effort for maintaining consistency with the RAD.
- Moreover, the ODD duplicates information in the source code and requires a high level of effort whenever the code changes. This often leads to an RAD and an ODD that are inaccurate or out of date.

- 2. ODD as extension of the RAD.** The second approach is to treat the object design model as an extension of the analysis model.
- In other terms, the object design is considered as the set of application objects augmented with solution objects.
 - The advantage of this solution is that maintaining consistency between the RAD and the ODD becomes much easier as a result of the reduction in redundancy.
 - The disadvantages of this solution include polluting the RAD with information that is irrelevant to the client and the user.
- Moreover, object design is rarely as simple as identifying additional solution objects. Often, application objects are changed or transformed to accommodate design goals or efficiency concerns.

3. ***ODD embedded into source code***. The third approach is to embed the ODD into the source code.

- Once the ODD becomes stable, we use the modeling tool to generate class stubs.
- We describe each class interface using tagged comments that distinguish source code comments from object design descriptions. We can then generate the ODD using a tool that parses the source code and extracts the relevant information.
- Once the object design model is documented in the code, we abandon the initial object design model.
- The advantage of this approach is that the consistency between the object design model and the source code is much easier to maintain:
 - when changes are made to the source code, the tagged comments are updated and the ODD regenerated. In this section, we focus only on this approach.